Habit Tracker — MVP Design & Self-Build Plan

Stack: Flutter (Android) · NestJS · PostgreSQL · Prisma **Goal:** Learning-first, finishable MVP that is publishable later. **Date:** 2026-06-23

---

1. Product Scope (MVP)

A simple daily habit tracker. Each user signs in, creates habits, and checks them off once per day. The app shows the current streak per habit and a history calendar.

In scope (v1): - Email + password accounts (JWT auth) - Create / rename / archive habits - Daily check-off (mark a habit done for a given day; un-check to undo) - Current streak per habit (computed) - History view (calendar / list of completed days) - Online-only (always talks to the backend)

Explicitly out of scope (later phases): - Reminders / notifications - Count/quantity habits and custom schedules - Offline-first sync - Social login, sharing, analytics dashboards

> **Why ruthless scope:** the fastest way to *learn the whole stack* is to ship one thin vertical slice end-to-end (auth → habit → check-off → streak → deploy), then layer features on.

---

2. Architecture

Online-only, three layers. The Flutter app holds **no business logic** beyond UI state — every read/write goes to the API. The API owns all rules (auth, ownership, streak computation) and is the only thing touching Postgres.

```
Flutter app (Android)  ──HTTPS/JSON──>  NestJS API  ──>  PostgreSQL
   - screens / UI                        - REST endpoints     - users
   - Riverpod state                      - JWT auth guards    - habits
   - dio api client                      - DTO validation     - habit_entries
   - secure token storage                - streak logic
```

Repo layout (one repo, two folders)

```
habit-tracker/
  backend/          # NestJS + Prisma
  mobile/           # Flutter app
  docs/             # this design + notes
  docker-compose.yml# local Postgres (+ optional backend)
  README.md
```

---

3. Data Model

Three tables. A "check-off" is just a row in `habit_entries`. Streaks are **computed from entries**, never stored, so there is nothing to keep in sync.

users

column	type	notes
id	uuid (PK)	default gen_random_uuid()
email	text	unique, lowercased
password_hash	text	bcrypt
created_at	timestamptz	default now()

habits

column	type	notes
id	uuid (PK)	
user_id	uuid (FK)	→ users.id, on delete cascade
name	text	required
color	text	optional hex, e.g. "#4F46E5"
created_at	timestamptz	default now()
archived_at	timestamptz	nullable; non-null = archived/hidden

habit_entries

column	type	notes
id	uuid (PK)	
habit_id	uuid (FK)	→ habits.id, on delete cascade
date	date	the day completed (no time component)
created_at	timestamptz	default now()

Constraint: `UNIQUE (habit_id, date)` — a habit can be completed at most once per day. Un-checking = delete the row.

Prisma schema (backend/prisma/schema.prisma)

```
generator client { provider = "prisma-client-js" }
datasource db { provider = "postgresql"; url = env("DATABASE_URL") }
```

```
model User {
  id           String   @id @default(uuid())
  email        String   @unique
  passwordHash String   @map("password_hash")
  createdAt    DateTime @default(now()) @map("created_at")
  habits       Habit[]
  @@map("users")
}

model Habit {
  id         String       @id @default(uuid())
  userId     String       @map("user_id")
  name       String
  color      String?
  createdAt  DateTime     @default(now()) @map("created_at")
  archivedAt DateTime?    @map("archived_at")
  user       User         @relation(fields: [userId], references: [id], onDelete: Cascade)
  entries    HabitEntry[]
  @@map("habits")
}

model HabitEntry {
  id        String   @id @default(uuid())
  habitId   String   @map("habit_id")
  date      DateTime @db.Date
  createdAt DateTime @default(now()) @map("created_at")
  habit     Habit    @relation(fields: [habitId], references: [id], onDelete: Cascade)
  @@unique([habitId, date])
  @@map("habit_entries")
}
```

---

4. Backend API (NestJS)

REST + JSON. All `/habits/**` routes require a valid JWT and enforce **ownership** (a user can only touch their own habits).

Auth

Method	Path	Body / notes	Returns
POST	`/auth/register`	`{ email, password }`	`{ user, accessToken, refreshToken }`
POST	`/auth/login`	`{ email, password }`	`{ user, accessToken, refreshToken }`
POST	`/auth/refresh`	`{ refreshToken }`	`{ accessToken, refreshToken }`
GET	`/auth/me`	(auth) returns current user	`{ user }`

- Passwords hashed with **bcrypt**.
- **Access token** short-lived (~15 min), **refresh token** long-lived (~30 days).
- Keep refresh simple for MVP: sign a refresh JWT; rotation/blacklist is a later hardening step (note it, don't build it now).

Habits

Method	Path	Body / query	Notes
GET	`/habits`	`?date=YYYY-MM-DD` (optional)	List active habits; if `date` given, include `doneToday` + `currentStreak`
POST	`/habits`	`{ name, color? }`	Create
PATCH	`/habits/:id`	`{ name?, color?, archived? }`	Rename / recolor / archive
DELETE	`/habits/:id`	—	Hard delete (cascades entries). Prefer archive in UI.

Entries (check-off)

Method	Path	Body / query	Notes
GET	`/habits/:id/entries`	`?from=YYYY-MM-DD&to=...`	Completed dates in range (for history)
POST	`/habits/:id/entries`	`{ date: YYYY-MM-DD }`	Check off a day (idempotent via unique constraint)
DELETE	`/habits/:id/entries/:date`	—	Un-check a day

Streak logic (server-side)

Given a habit's set of completed `date`s and "today": - **currentStreak** = number of consecutive days ending at today (or yesterday, if today not yet done) with an entry. - Decide and document the rule: *today not-yet-done does NOT break the streak* — count back from yesterday; if today is done, include it. - Implement as a pure function `computeCurrentStreak(dates: Date[], today: Date): number` so it is **unit-testable in isolation**.

Module structure

```
backend/src/
  auth/        # controller, service, jwt strategy, guards, dto
  users/       # service (lookup/create), prisma access
  habits/      # controller, service, dto
  entries/     # controller (nested under habits), service
  common/      # exception filter, interceptors, decorators (@CurrentUser)
  prisma/      # PrismaService (module-wide)
  main.ts
```

---

5. Mobile App (Flutter)

State: Riverpod. **HTTP:** dio. **Token storage:** flutter_secure_storage. **Routing:** go_router.

Folder structure

```
mobile/lib/
  main.dart
  core/
    api/          # dio client + auth interceptor (401 -> refresh -> retry)
    storage/      # secure storage wrapper for tokens
    theme/
  models/         # User, Habit, HabitEntry (fromJson/toJson)
  features/
    auth/         # login & register screens + auth provider/repository
    habits/       # today list, add/edit habit, habit detail/history
  widgets/        # shared widgets (habit tile, streak chip, empty state)
```

Screens

1. **Login / Register** — email + password forms with validation; on success store tokens, go to Today.
2. **Today (home)** — list of active habits with a check toggle and a streak chip. Tap toggle → POST/DELETE entry → optimistic UI update.
3. **Add / Edit Habit** — name + optional color; archive button on edit.
4. **Habit Detail / History** — month calendar (e.g. `table_calendar`) highlighting completed days + current streak.

API client behavior

- Attach `Authorization: Bearer <access>` on every request.
- On `401`: call `/auth/refresh`; on success retry the original request; on failure clear tokens and route to Login.
- Show loading + error states (snackbars); no silent failures.

Suggested packages

`flutter_riverpod`, `dio`, `flutter_secure_storage`, `go_router`, `table_calendar`, `intl`.

---

6. Error Handling & Validation

Backend - DTOs validated with `class-validator` + global `ValidationPipe` (`whitelist: true`). - Global **exception filter** → consistent JSON: `{ statusCode, message, error }`. - Guards: `JwtAuthGuard` (auth) + ownership check in habit/entry services (404 if not owner, to avoid leaking existence). - Map known cases: duplicate email → 409; bad credentials → 401; missing habit → 404.

Mobile - Central dio error mapping → friendly messages. - Form-level validation before sending. - Distinguish: network error vs auth error vs validation error.

---

7. Testing

Keep it focused — test the logic that's easy to get wrong.

Backend (Jest) - **Unit:** `computeCurrentStreak` (edge cases: empty, single day, gap, today done vs not, across month boundary). Auth service (hash/verify, token issue). - **e2e (supertest):** register → login → create habit → check off → list shows streak → uncheck. One happy-path flow + a couple of auth failures.

Mobile - Widget tests for Login form validation and the Today habit tile toggle. - A unit test for any client-side date formatting helper.

> Aim: green tests for the streak function and the auth+habit e2e flow before deploying. That's enough confidence for an MVP.

---

8. Deployment

Local dev - `docker-compose.yml` runs Postgres (and optionally the backend). - Backend: `npm run start:dev`; run `prisma migrate dev` for schema. - Flutter: run on Android emulator / physical device. Point the app at the backend via a `--dart-define=API_BASE_URL=...` build flag (use your machine's LAN IP for a physical device).

Backend hosting - Dockerize the NestJS app. - Deploy to **Railway** or **Render** (both have managed Postgres + simple Git deploys — good for a first deploy). - Set env: `DATABASE_URL`, `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, token TTLs. Run migrations on deploy.

Android delivery - Generate a signing **keystore**; configure `key.properties` + `build.gradle`. - `flutter build apk --release` (or `appbundle`) with `--dart-define=API_BASE_URL=<your prod URL>`. - For yourself/testers: install the signed APK directly. - For Play Store later: Google Play developer account (one-time $25), upload the `.aab`, fill store listing. (Out of MVP scope — note for later.)

---

9. Build Plan — Phased Checklist

Work top to bottom. Each phase is a vertical step you can verify before moving on.

Phase 0 — Project setup

- [ ] Create repo `habit-tracker/` with `backend/`, `mobile/`, `docs/`.
- [ ] `docker-compose.yml` with Postgres; verify you can connect.
- [ ] `nest new backend`; add Prisma; `prisma init`; write schema (Section 3); `prisma migrate dev`.
- [ ] `flutter create mobile`; add packages (Section 5).
- **Done when:** backend boots, DB migrated, Flutter runs the starter app on a device.

Phase 1 — Backend auth

- [ ] Users + Auth modules; register/login with bcrypt; JWT access+refresh; `JwtAuthGuard`; `/auth/me`.
- [ ] `@CurrentUser` decorator; global ValidationPipe + exception filter.
- **Done when:** you can register, login, and call `/auth/me` with the token (test via curl/Postman).

Phase 2 — Backend habits + entries + streak

- [ ] Habits CRUD with ownership; Entries check-off/uncheck with unique constraint.
- [ ] `computeCurrentStreak` pure function; `GET /habits?date=` returns `doneToday` + `currentStreak`.
- **Done when:** full happy-path works via Postman.

Phase 3 — Backend tests & hardening

- [ ] Unit tests for streak + auth; e2e happy-path flow.
- [ ] Consistent error responses (409/401/404 cases).
- **Done when:** `npm test` is green.

Phase 4 — Flutter foundation + auth

- [ ] dio client + auth interceptor (401→refresh→retry); secure token storage.
- [ ] Login/Register screens + auth provider; go_router with auth redirect.
- **Done when:** you can log in on-device and land on an (empty) Today screen.

Phase 5 — Flutter Today screen

- [ ] Fetch `/habits?date=today`; render tiles with streak chip + toggle.
- [ ] Optimistic check/uncheck wired to entries endpoints.
- **Done when:** checking a habit persists and the streak updates after refresh.

Phase 6 — Flutter habit management + history

- [ ] Add/Edit/Archive habit screens.
- [ ] Habit detail with month calendar of completed days.
- **Done when:** full CRUD + history works on-device.

Phase 7 — Polish & tests

- [ ] Loading/empty/error states; basic theming; pull-to-refresh.
- [ ] Widget tests for login validation + toggle.
- **Done when:** no obvious dead-ends; tests green.

Phase 8 — Deploy

- [ ] Dockerize backend; deploy to Railway/Render with managed Postgres; run migrations.
- [ ] Point app at prod via `--dart-define`; build signed release APK; install & smoke-test.
- **Done when:** the release APK on your phone talks to the deployed backend.

---

10. Environment Variables

backend/.env

```
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/habit
JWT_ACCESS_SECRET=<random-long-string>
JWT_REFRESH_SECRET=<different-random-long-string>
JWT_ACCESS_TTL=15m
```

```
JWT_REFRESH_TTL=30d
PORT=3000
```

mobile — pass at build/run time:

```
flutter run --dart-define=API_BASE_URL=http://<LAN-IP>:3000
```

> Never commit `.env` or the keystore. Add them to `.gitignore`.

---

11. Learning Order Tips

- Build **backend before mobile** — you can test every endpoint with Postman before any UI exists, which isolates bugs.
- Get **one habit fully working** (create → check → streak) before building the nicer screens.
- Treat `computeCurrentStreak` as your first unit-tested function — it teaches the testing loop on a small, pure target.
- Deploy **early** (even a bare backend) so the final deploy phase isn't a surprise.

---

12. Future Phases (post-MVP, not now)

1. Local reminder notifications (`flutter_local_notifications`).
2. Count/quantity habits and per-habit schedules (e.g. Mon/Wed/Fri).
3. Offline-first with local SQLite + sync.
4. Google Sign-In; refresh-token rotation/revocation.
5. Stats/insights; Play Store release.

Open Questions (decide before/while building)

- **Day boundary / timezone:** which timezone defines "today" for streaks — device local or a fixed server tz? Recommended: send the client's local date in requests and compute streaks against that. Confirm before Phase 2.
- **Delete vs archive in UI:** plan keeps both endpoints; recommend exposing only *archive* in the UI to preserve history.